

TEK4090

Introduction to Modern Control

Lecture 4: Optimal Control

Mathias Hudoba de Badyn

September 25, 2024

Plan for Rest of the Course

- ▶ Shift lectures by one
- ▶ Either record the last lecture, or find an alternate day
- ▶ Mario will record the Matlab tutorial
- ▶ Homework 1 grades: Monday; Homework 2: out Friday



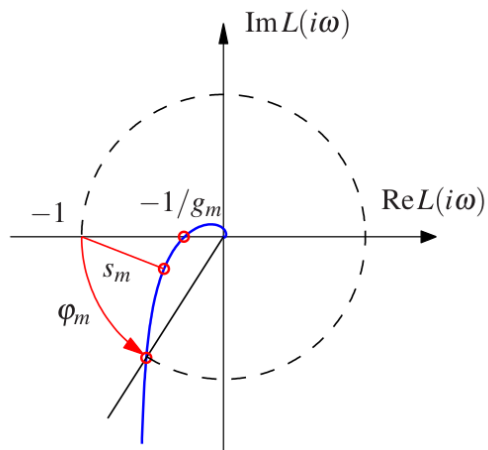
Master Thesis Topics + Final Project Topics

- ▶ Master's thesis topics will be available on the Cybernetics web page
- ▶ Homework 2 will include a portion on planning your final project
- ▶ As part of (or shortly after) Homework 2, you should set up a zoom/office meeting with me or Mario ($\approx 15\text{--}30$ mins) to go over your project plan

Reading Assignment

1. **Lewis, Syrmos, Vraibie:** Ch2,3 & 6. Warning: typos galore

Recap from Last Time



- ▶ Nyquist stability and Bode Plots
- ▶ Stability margins
- ▶ Introduction to Linear Systems

Clarification from Last Time

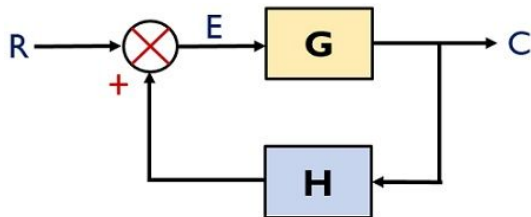
I made some comments about non-minimum phase system and RHP zeros

Non-Minimum Phase



Open-loop zeros of transfer function from u to y

Stability

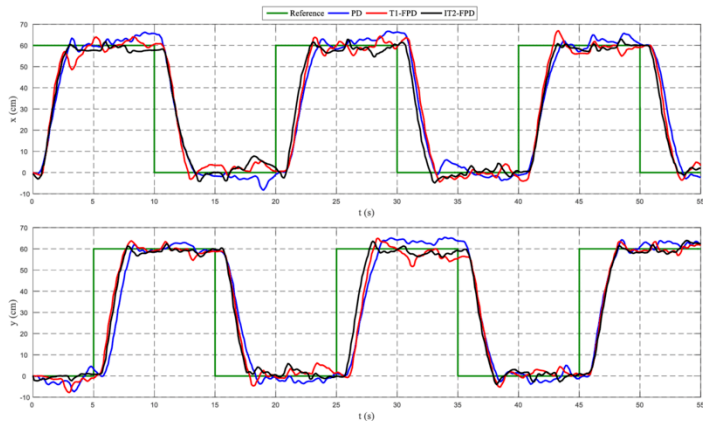


$$\frac{C}{R} = \frac{G}{1 - GH}$$

Closed-loop zeros of transfer function from c to e

Motivation for Stabilization

System is stable $\implies x(t) \rightarrow r(t)$



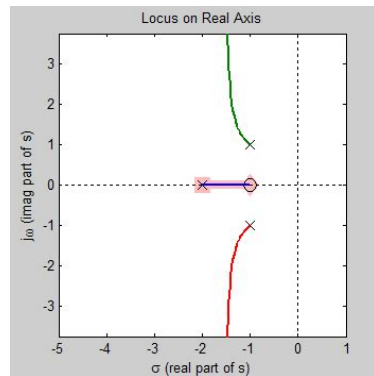
Stabilization by State Feedback

Stabilization by State Feedback

Motivation for Optimal Control

But how do we even pick the gains properly?

- ▶ Potentially by staring at the Bode plots and making sure the frequency response fits some criteria
- ▶ **Root Locus:** plot the 'trajectory' of the poles in the complex plane as a function of k_i 's



Motivation for Optimal Control

Control is the math behind automation. Why don't we automate picking the gains?



Just pick the best one



Motivation for Optimization

Ok, but, how?



- ▶ General recipe for optimal control:
- ▶ Define a performance metric for your system
- ▶ Do some calculus to optimize the metric
- ▶ Give up and do machine learning?

basics of (un)constrained optimization

Unconstrained optimization

► unconstrained optimization:

$$\underset{x \in \mathbb{R}^n}{\text{minimize}} f(x) \quad (1)$$

where $f: \mathbb{R}^n \rightarrow \mathbb{R}$ is a continuously differentiable *objective* function

► $x^* \in \mathbb{R}^n$ is said to be a **minimizer** and $f(x^*)$ is a **minimum** if

$$f(x^*) \leq f(x) \quad \text{for all } x \in \mathbb{R}^n$$

► a minimizer and minimum are said to be **strict** if

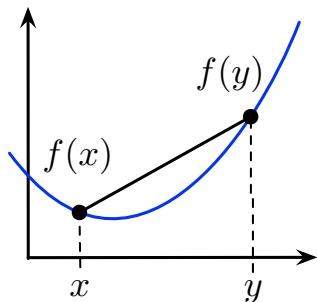
$$f(x^*) < f(x) \quad \text{for all } x \in \mathbb{R}^n \setminus \{x^*\}.$$

Note: all concepts are defined globally, but there are also local versions

► on **gradients**: $\frac{\partial f}{\partial x}: \mathbb{R}^n \rightarrow \mathbb{R}^n$ is the *column vector* of partial derivatives

Convexity

- f is said to be **convex** if “function is below line segment”



$$\forall x, y \in \mathbb{R}^n \text{ and } \forall \alpha, \beta \geq 0 \text{ with } \alpha + \beta = 1$$

$$f(\alpha x + \beta y) \leq \alpha f(x) + \beta f(y)$$

⇒ f is **strictly convex** if inequality is strict

⇒ f is **concave** if $-f$ is convex

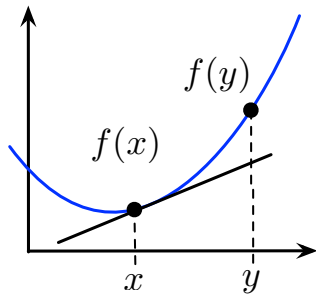
Convexity

- **underestimator property**: “convex iff function above its tangents”

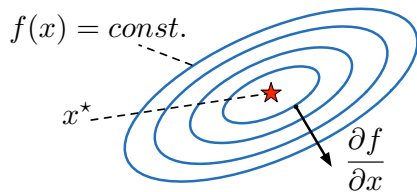
a cont. diff. function f is convex
if and only if for all $x, y \in \mathbb{R}^n$

$$f(y) \geq f(x) + (y - x)^\top \frac{\partial f(x)}{\partial x}$$

(strictly convex if inequality is strict for $x \neq y$)

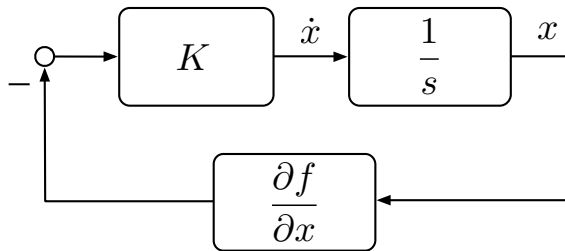


Unconstrained optimization & gradient flows



geometry of an unconstrained optimization problem

$$\dot{x} = -K \frac{\partial f(x)}{\partial x}$$



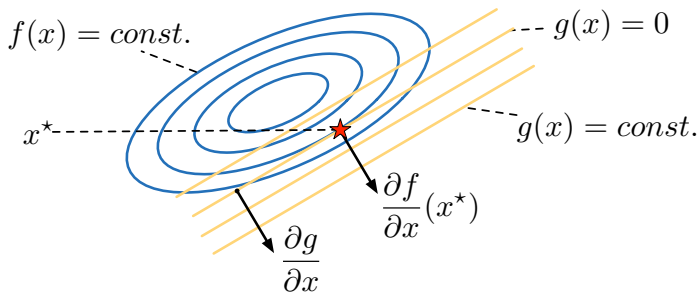
block-diagram of negative gradient flow with positive definite gain $K \in \mathbb{R}^{n \times n}$

Constrained optimization

► linearly-constrained optimization:

$$\begin{array}{ll} \text{minimize}_{x \in \mathbb{R}^n} & f(x) \\ \text{s.t.} & g(x) = \mathbf{0} \end{array} \quad (2)$$

where f is continuously differentiable & $g(x) = Ax - b$ is **linear-affine**



$\Rightarrow$ minimizer x^* when $\{g(x) = \mathbf{0}\}$ **tangent** to $\{f(x) = \text{const.}\} \Leftrightarrow \frac{\partial f}{\partial x} \parallel \frac{\partial g}{\partial x}$

Constrained Optimization

► linearly-constrained optimization:

$$\begin{array}{ll} \text{minimize}_{x \in \mathbb{R}^n} & f(x) \\ \text{s.t.} & g(x) = \mathbf{0} \end{array} \quad (3)$$

where f is continuously differentiable & $g(x) = Ax - b$ is linear-affine

⇒ minimizer x^* satisfies **KKT conditions** (after Karush, Kuhn, Tucker):

KKT Conditions

(i) constraint equation: $\mathbf{0} = g(x) = Ax - b$

(ii) tangency condition: $\mathbf{0} = \frac{\partial f(x)}{\partial x} + \frac{\partial g(x)}{\partial x}^\top \lambda,$

where λ is Lagrange multiplier or *dual variable* (analogous: x is *primal variable*)

Constrained Optimization

► linearly-constrained optimization:

$$\begin{array}{ll} \text{minimize}_{x \in \mathbb{R}^n} & f(x) \\ \text{s.t.} & g(x) := Ax - b = \mathbf{0} \end{array} \quad (4)$$

KKT Conditions

(i) constraint equation: $\mathbf{0} = g(x) = Ax - b$

(ii) tangency condition: $\mathbf{0} = \frac{\partial f(x)}{\partial x} + \frac{\partial g(x)}{\partial x}^\top \lambda$

1. the KKT conditions are necessary conditions for optimality of x^*
2. if f is convex, then the KKT conditions are sufficient for optimality, and their solutions (x^*, λ^*) specify all minimizers and optimal dual variables
3. if f is strictly convex and A has full rank, then the KKT conditions admit only a single solution (x^*, λ^*)

Constrained Optimization

► linearly-constrained optimization:

$$\begin{array}{ll} \text{minimize}_{x \in \mathbb{R}^n} & f(x) \\ \text{s.t.} & g(x) := Ax - b = \mathbf{0} \end{array} \quad (5)$$

where f is cont. diff. & **convex** and $g(x) = Ax - b$ is linear-affine

⇒ minimizer x^* and multiplier λ^* from **KKT conditions**:

(i) constraint equation: $\mathbf{0} = g(x) = Ax - b$

(ii) tangency condition: $\mathbf{0} = \frac{\partial f(x)}{\partial x} + \frac{\partial g(x)}{\partial x}^\top \lambda$

► **Lagrangian**: $\mathcal{L}(x, \lambda) = f(x) + \lambda^\top g(x)$ is convex in x and concave in λ

⇒ $\mathcal{L}(x, \lambda)$ has **saddle points**: $\mathcal{L}(x^*, \lambda) \leq \mathcal{L}(x^*, \lambda^*) \leq \mathcal{L}(x, \lambda^*)$

⇒ saddle points = { minimizer x^* and multiplier λ^* } from KKT:

$$\mathbf{0} = \frac{\partial \mathcal{L}(x, \lambda)}{\partial x} = \frac{\partial f(x)}{\partial x} + \frac{\partial g(x)}{\partial x}^\top \lambda \quad \mathbf{0} = \frac{\partial \mathcal{L}(x, \lambda)}{\partial \lambda} = g(x)$$

Linear Quadratic Case

Linear Quadratic Case

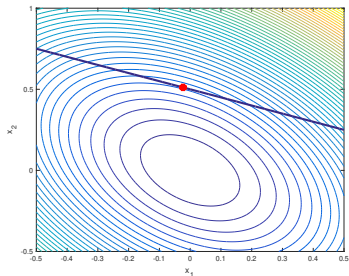
Running example for linear quadratic (LQ) case

$$\underset{x \in \mathbb{R}^n}{\text{minimize}} \ f(x) = \frac{1}{2} (x - \underline{x})^\top P (x - \underline{x}) \quad g(x) = Ax - b = 0$$

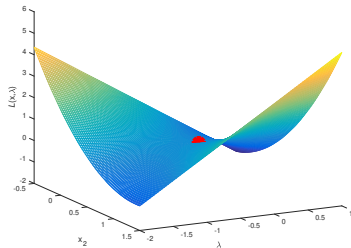
data: $P = \begin{bmatrix} 2.6 & 0.8 \\ 0.8 & 1.4 \end{bmatrix}$, $\underline{x} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$, $A = \begin{bmatrix} 1 & 2 \end{bmatrix}$, $b = 1$

facts: P is positive definite with eigenvalues $\{1, 3\}$, optimizer

$$x^* = [-0.0233 \ 0.5116]^\top, \text{ and optimal multiplier } \lambda^* = -0.3488$$



geometry of the LQ problem



Lagrangian projected on $x_1 = 0$

Primal-Dual Saddle-Point Flow

Primal-Dual Saddle-Point Flow

Useful Matrix Lemma

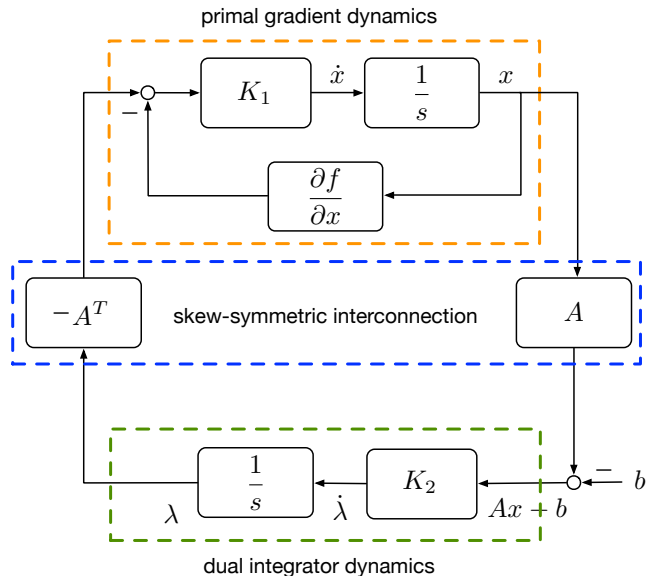
Lemma: Consider a positive semidefinite matrix $P \in \mathbb{R}^{n \times n}$ and a matrix $A \in \mathbb{R}^{m \times n}$ with $n \geq m$ forming the composite **saddle matrix**

$$\mathcal{A} = \begin{bmatrix} -P & -A^\top \\ A & \mathbf{0}_{m \times n} \end{bmatrix}.$$

The matrix $\mathcal{A}$ has the following properties:

- 1) all eigenvalues are in the closed left half-plane: $\text{spec}(\mathcal{A}) = \{\lambda \in \mathbb{C} \mid \mathcal{R}(\lambda) \leq 0\}$. Moreover, all eigenvalues on the imaginary axis have equal algebraic and geometric multiplicity;
- 2) if $\ker(P) \cap \text{image}(A^\top) \subseteq \{\mathbf{0}_n\}$, then $\mathcal{A}$ has no eigenvalues on the imaginary axis except for 0; and
- 3) if P is positive definite and A has full rank, then $\mathcal{A}$ has no eigenvalues on the imaginary axis.

Block-diagram of primal-dual saddle-point flow



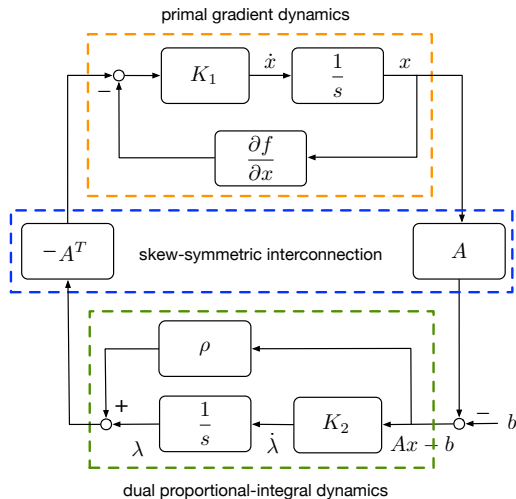
$$\dot{x} = -K_1 \frac{\partial f(x)}{\partial x} - K_1 A^T \lambda$$

$$\dot{\lambda} = K_2 (Ax - b)$$

1. primal gradient descent
2. dual integral control penalizing constraint violation
3. skew-symmetric interconnection

Augmented Primal-Dual Saddle-Point Flow

Block-diagram of augmented primal-dual saddle-point flow



$$\dot{x} = -K_1 \frac{\partial f(x)}{\partial x} - K_1 A^T \lambda$$

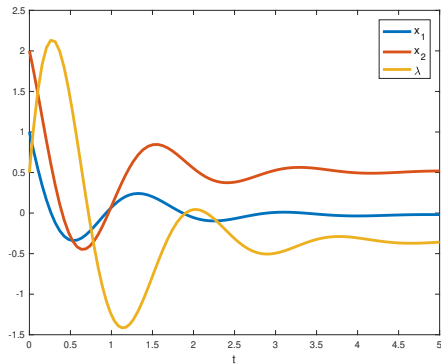
$$- \rho K_1 A^T (Ax - b)$$

$$\dot{\lambda} = K_2 (Ax - b)$$

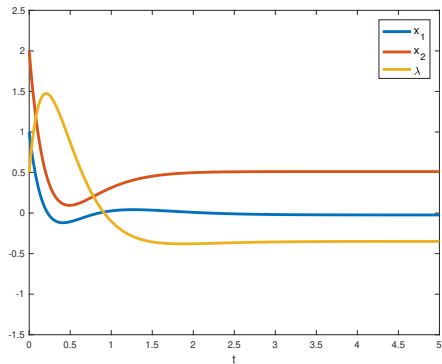
1. primal gradient descent
2. dual proportional-integral control penalizing constraint violation
3. skew-symmetric interconnection

Linear-Quadratic Case

Standard and augmented saddle-point flow for LQ program



standard saddle-point flow



augmented saddle-point flow ($\rho = 1$)

augmentation induces (stricter) convexity & better performance (damping)

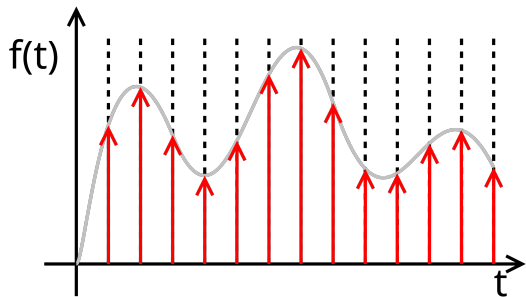
Punchline + Caveat

- ▶ Algorithms (or rather, dynamics) find the optimal point
- ▶ Caveat: nobody does this in continuous time...

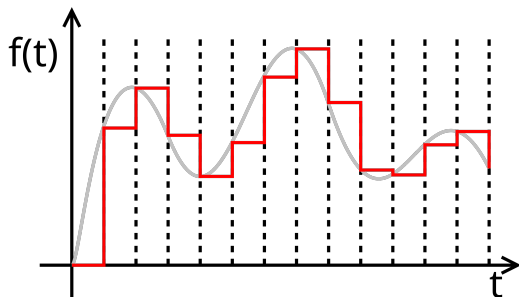
Optimal Control

Discrete-Time Systems

This stuff is a bit more intuitive in **discrete time**, so let's introduce that here.



Sampling



Sample-and-hold

Discrete-Time Systems

$$\begin{aligned}\hat{x}_{k+1} &= \bar{A}\hat{x}_k + \bar{B}\hat{u}_k \\ \hat{y}_k &= \bar{C}\hat{x}_k + \bar{D}\hat{u}_k\end{aligned}$$

where the discrete-time matrices are (for a timestep Δt):

$$\begin{aligned}\bar{A} &= e^{A\Delta t}, \quad \bar{B} = A^{-1} \left(e^{A\Delta t} - I \right) B \\ \bar{C} &= C, \quad \bar{D} = D.\end{aligned}$$

Optimal Control

Static Optimization

$$\begin{array}{ll} \min_{x \in \mathbb{R}^n} & \frac{1}{2} x^T P x \\ \text{s.t.} & A x = b \end{array}$$

Dynamic Optimization

$$\begin{array}{ll} \min_{x \in \mathbb{R}^n \times \mathcal{T}} & \frac{1}{2} \int_{\mathcal{T}} [x^T P x + u^T R u] dt \\ \text{s.t.} & \dot{x} = A x + B u \end{array}$$

Optimal **points** versus optimal **trajectories**

Optimal Control

Static Optimization

$$\begin{array}{ll} \min_{x \in \mathbb{R}^n} & \frac{1}{2} x^T P x \\ \text{s.t.} & A x = b \end{array}$$

Dynamic Optimization

$$\begin{array}{ll} \min_{x \in \mathbb{R}^n \times [0, T]} & \frac{1}{2} \sum_{k=1}^T [x_k^T P_k x_k + u_k^T R_k u_k] \\ \text{s.t.} & x_{k+1} = \bar{A} x_k + \bar{B} u_k \end{array}$$

Optimal **points** versus optimal **sequence**

Bellman's Principle of Optimality

- ▶ The optimal trajectory from where you are doesn't depend on where you've been
- ▶ Eg., sunken cost fallacy
- ▶ Dynamic programming: start at the end, go back, optimize 'cost-to-go'

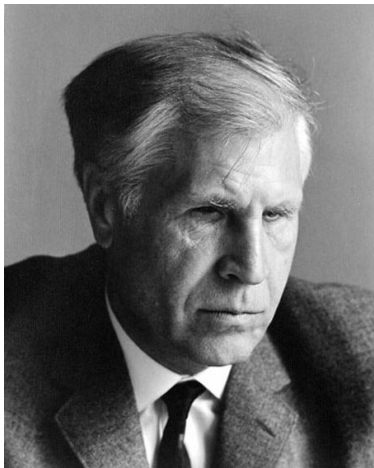
Discrete-Time LQR

Discrete-Time LQR

Discrete-Time LQR

Pontryagin Maximum Principle

What about the **KKT conditions** for **dynamic problems**?



Super general problem:

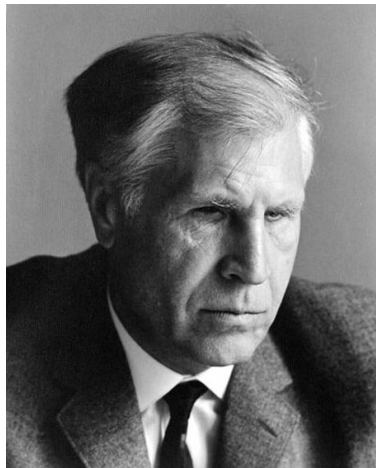
$$\begin{aligned} \min \quad & \phi(x(T), T) + \int_0^T L(x, u, t) dt \\ \text{s.t.} \quad & \psi(x(T), T) = 0 \\ & \dot{x} = f(x, u, t) \end{aligned}$$

Define the **Hamiltonian**:

$$H(x, u, \lambda, t) := L(x, u, \lambda, t) + \lambda(t)^T f(x, u, t)$$

Pontryagin Maximum Principle

What about the **KKT conditions** for **dynamic problems**?



$$H(x, u, \lambda, t) := L(x, u, \lambda, t) + \lambda(t)^T f(x, u, t)$$

Pontryagin's maximum principle:

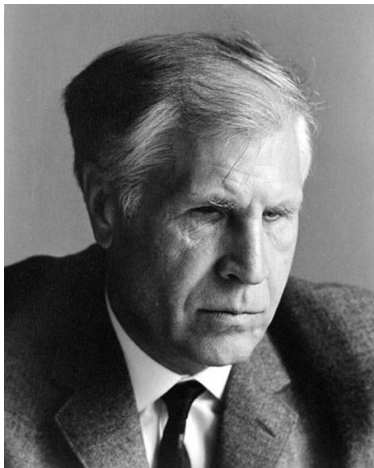
$$H(x^*, u^*, \lambda^*, t) \leq H(x^*, u, \lambda^*, t), \quad \forall t \in [0, T]$$

Coupled DEs:

$$\begin{aligned}\dot{x} &= \frac{\partial H}{\partial \lambda} = f \\ -\dot{\lambda} &= \frac{\partial H}{\partial x} = \frac{\partial f^T}{\partial x} \lambda + \frac{\partial L}{\partial x}\end{aligned}$$

Pontryagin Maximum Principle

What about the **KKT conditions** for **dynamic problems**?



Coupled DEs:

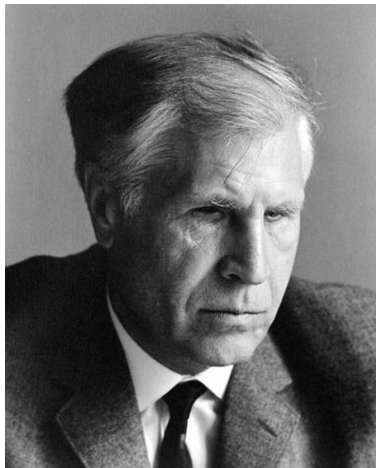
$$\begin{aligned}\dot{x} &= \frac{\partial H}{\partial \lambda} = f \\ -\dot{\lambda} &= \frac{\partial H}{\partial x} = \frac{\partial f^T}{\partial x} \lambda + \frac{\partial L}{\partial x}\end{aligned}$$

Boundary conditions:

$$0 = \frac{\partial H}{\partial u} = \frac{\partial L}{\partial u} + \frac{\partial f^T}{\partial u} \lambda$$

Pontryagin Maximum Principle

What about the **KKT conditions** for **dynamic problems**?



Coupled DEs:

$$\begin{aligned}\dot{x} &= \frac{\partial H}{\partial \lambda} = f \\ -\dot{\lambda} &= \frac{\partial H}{\partial x} = \frac{\partial f^T}{\partial x} \lambda + \frac{\partial L}{\partial x}\end{aligned}$$

Boundary conditions:

$$(\phi_x + \psi_x^T v - \lambda)^T \Big|_T dx(T) + (\phi_t + \psi_t^T v + H) \Big|_T dT = 0$$

Continuous-Time LQR vs Discrete-Time LQR

Discrete-Time LQR

$$\begin{aligned} \min_{x \in \mathbb{R}^n \times [0, T]} \quad & \frac{1}{2} \sum_{k=1}^T [x_k^T P_k x_k + u_k^T R_k u_k] \\ \text{s.t.} \quad & x_{k+1} = \bar{A} x_k + \bar{B} u_k \end{aligned}$$

Solution:

$$\begin{aligned} J_k^* &= \frac{1}{2} x_k^T S_k x_k \\ S_k &= \bar{A}^T S_{k+1} \bar{A} - \bar{A}^T S_{k+1} \bar{B} K_k + P_k \\ K_k &= (B^T S_{k+1} B + R_k)^{-1} B^T S_{k+1} \bar{A} \\ u_k^* &= -K_k x_k \end{aligned}$$

Continuous-Time LQR

$$\begin{aligned} \min_{x \in \mathbb{R}^n \times \mathcal{T}} \quad & \frac{1}{2} \int_{\mathcal{T}} [x^T P x + u^T R u] dt \\ \text{s.t.} \quad & \dot{x} = Ax + Bu \end{aligned}$$

Solution:

$$\begin{aligned} J^*(t) &= \frac{1}{2} x(t)^T S(t) x(t) \\ -\dot{S} &= SA + A^T S - SBR^{-1}B^T S + Q \\ K &= -R^{-1}B^T S \\ u &= -Kx \end{aligned}$$

Motivation for MPC

In practice, LQR can tune gains really nicely

Some cons:

- ▶ Cannot handle constraints
- ▶ Computationally intensive (finite time-horizon)
- ▶ No guaranteed margins when noise in system

Guaranteed Margins for LQG Regulators

JOHN C. DOYLE

Abstract—There are none.

Motivation for MPC

$$\begin{aligned} \min_{x \in \mathbb{R}^n \times [0, T]} \quad & \frac{1}{2} \sum_{k=1}^T [x_k^T P_k x_k + u_k^T R_k u_k] \\ \text{s.t.} \quad & x_{k+1} = \bar{A}x_k + \bar{B}u_k \\ & x_k \in \mathcal{X}_k \\ & u_k \in \mathcal{U}_k \end{aligned}$$



CVXPY

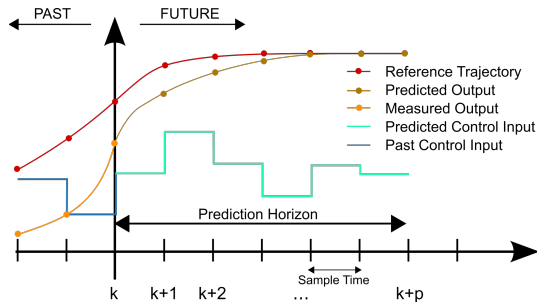
Model Predictive Control

MPC

- ▶ At each timestep k solve:

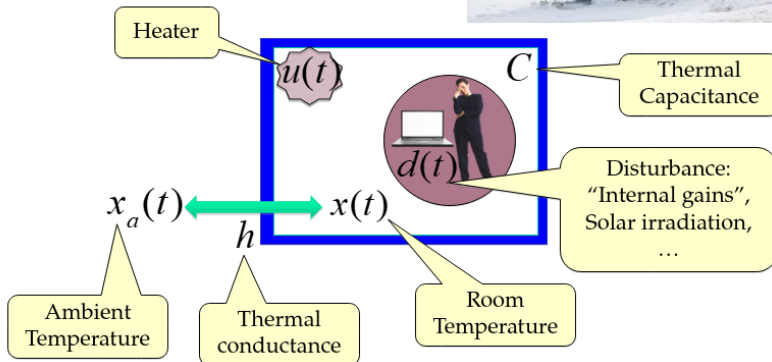
$$\begin{aligned} \min \quad & \sum_{t=k}^{k+p} c_k(x_k, u_k) \\ \text{s.t.} \quad & x_{t+1} = f(x_t, u_t) \\ & x_k = x(k) \\ & x_t \in \mathcal{X} \\ & u_t \in \mathcal{U} \end{aligned}$$

- ▶ Apply the first control input $u(k) = u_k$
- ▶ $k \mapsto k+1$
- ▶ Repeat



Building Control

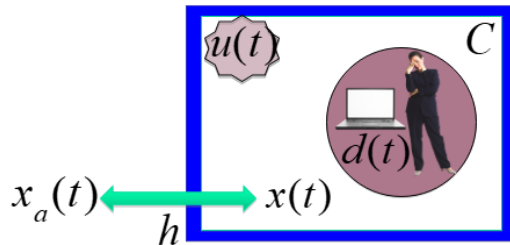
Single room house



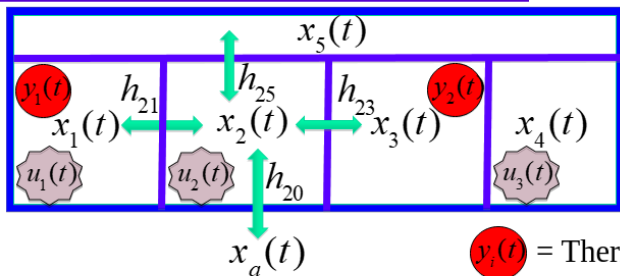
Modeling Buildings – Single Room

$$C \frac{d}{dt} x(t) = h [x_a(t) - x(t)] + u(t) + d(t)$$

- ▶ C : Thermal capacity of the room
- ▶ h : Insulation of the wall
- ▶ $u(t)$: Heat input at time t



Multiple zones



$y_i(t)$ = Thermometer
 $u_i(t)$ = Radiator

$$C_2 \dot{x}_2(t) = h_{20}(x_a(t) - x_2(t)) + h_{21}(x_1(t) - x_2(t)) + h_{23}(x_3(t) - x_2(t)) + h_{25}(x_5(t) - x_2(t)) + u_2(t) + d_2(t)$$

Modeling Buildings – Multiple Zones

$$x(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \\ x_3(t) \\ x_4(t) \\ x_5(t) \end{bmatrix}, \quad u(t) = \begin{bmatrix} u_1(t) \\ u_2(t) \\ u_3(t) \end{bmatrix}, \quad d(t) = \begin{bmatrix} d_1(t) \\ d_2(t) \\ d_3(t) \\ d_4(t) \\ d_5(t) \end{bmatrix} \quad (6)$$

$$H_{ij} = \begin{cases} \frac{h_{ij}}{\bar{C}_i} & \text{if } i \text{ is a neighbour of } j \\ 0 & \text{else} \end{cases} \quad (7)$$

$$H_{ii} = \sum_{j=1, \dots, 5} H_{ij} \quad (8)$$

Modeling Buildings – Multiple Zones

$$\dot{x}(t) = \begin{bmatrix} -H_{11} & H_{12} & 0 & 0 & H_{15} \\ H_{21} & -H_{22} & H_{23} & 0 & H_{25} \\ 0 & H_{32} & H_{33} & H_{34} & H_{35} \\ 0 & 0 & H_{43} & -H_{44} & H_{45} \\ H_{51} & H_{52} & H_{53} & H_{54} & -H_{55} \end{bmatrix} x(t) + \begin{bmatrix} H_{10} \\ H_{20} \\ H_{30} \\ H_{40} \\ H_{50} \end{bmatrix} x_a(t) + \quad (9)$$

$$\begin{bmatrix} \frac{1}{C_1} & 0 & 0 \\ 0 & \frac{1}{C_2} & 0 \\ 0 & 0 & 0 \\ 0 & 0 & \frac{1}{C_4} \\ 0 & 0 & 0 \end{bmatrix} u(t) + \begin{bmatrix} \frac{1}{C_1} & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{C_2} & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{C_3} & 0 & 0 \\ 0 & \frac{1}{C_4} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \frac{1}{C_5} \end{bmatrix} d(t) \quad (10)$$

Modeling Buildings – Multiple Zones

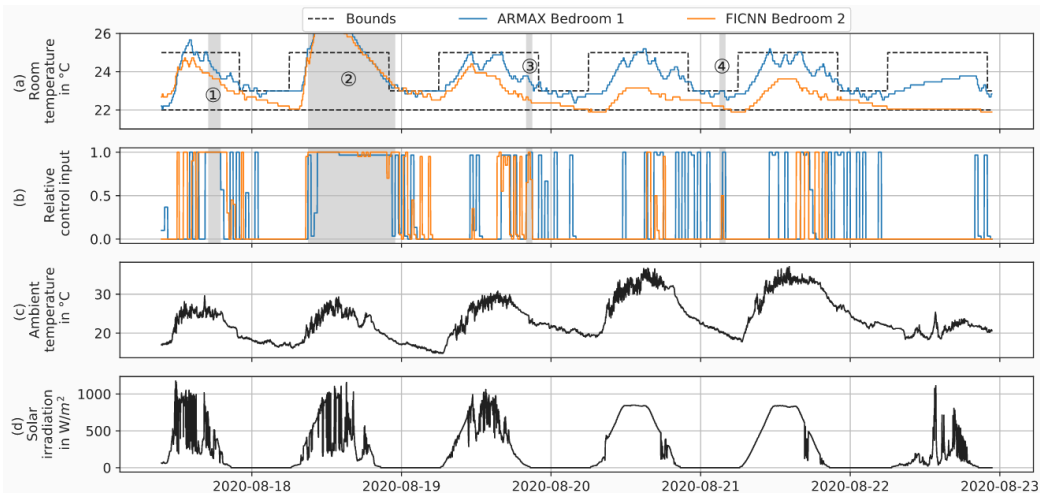
$$\dot{x} = Ax(t) + bx_a(t) + Bu(t) + Dd(t) \quad (11)$$

$$y(t) = Cx(t) + cx_a(t) \quad (12)$$

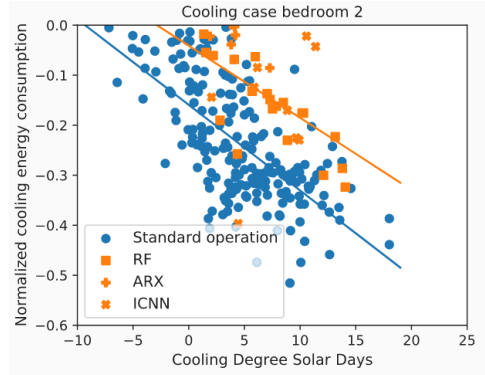
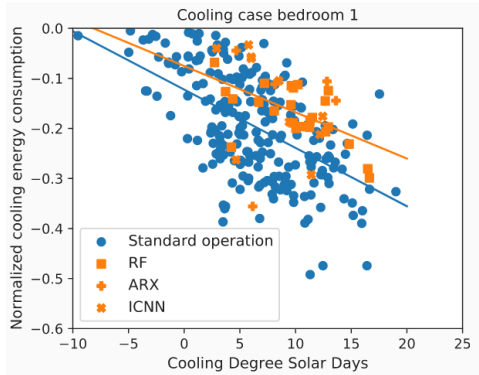
Example of Building MPC



Performance of ICNNs In the Loop



Comparison with other ML Techniques + Hysteresis Control



Punchline: MPC consumes 33% less cooling energy at 5 CDSD and 28% less at 15 CDSD.

Next Time



- ▶ Observability (“Where am I?” from sensors)
- ▶ Kalman Filter (more black magic)
- ▶ Spacecraft GNC